

00 — Vision & Philosophy

Objetivo de esta sección: explicar *por qué existe Puglit* y cuáles son sus principios innegociables — la constitución filosófica contra la que se mide cada decisión técnica del resto del whitepaper. No describe *cómo* se construye un app (eso es trabajo de las secciones posteriores), sino *qué creemos y qué nos negamos a ser*.

Executive Summary

Qué es Puglit

Puglit es un sistema **genético multi-agente, open-source, self-hosted y agent-first** que genera aplicaciones SaaS full-stack completas a partir de una idea de una línea. El stack de salida es fijo y opinado: **Next.js 16 (App Router, Turbopack) + Postgres crudo vía pg**, con el resto de la "infraestructura aburrida" (auth, rate-limit, analytics, i18n, mailer, pool de DB) ensamblada determinísticamente desde el spine ([spine/](#), ver [_SHARED_FACTS.md](#) y [README.md](#)).

La afirmación central, repetida palabra por palabra en el header de [web/lib/app-builder.ts](#), es que **nada está hardcodeado a un producto específico**: el blueprint agent infiere las superficies y operaciones a partir de las entidades y el pitch. El mismo motor que produce un clon de Airbnb produce un juego retro de Atari o un tracker de hábitos.

Puglit corre **100% local sobre Ollama** — sin API key, sin costo por token — y permite **BYO (Bring Your Own)** de cualquier proveedor compatible con OpenAI o con Anthropic, mezclando proveedores por tier ([web/lib/openai.ts](#), env [PUGLIT_MODEL_*](#)).



El problema que resuelve

Como lo plantea el [README.md](#) : la mayoría de los AI builders (v0, Bolt, Lovable) te dan un **frontend bonito** — y después pasás semanas cableando el 80% aburrido-pero-crítico: auth, suscripciones, email transaccional, cron jobs, panel de admin, capa de IA, deploy. Ese 80% es mecánico y está probado, pero es exactamente donde los generadores basados en LLM alucinan y fallan.

Puglit invierte la ecuación. El 80% mecánico se **ensambla determinísticamente** desde un spine probado en producción (extraído de un producto real con revenue real). El LLM se reserva para el 20% genuinamente único: las entidades, el schema, los prompts, el copy y el "engine" propio del producto. El resultado declarado: **la confiabilidad de un boilerplate con la flexibilidad de un generador**.

Público objetivo

Audiencia	Qué obtiene
Solo-founders / indie hackers	Un SaaS monetizable y desplegable sin re-decidir el stack cada vez.
Equipos de producto	Un piso determinístico (auth/pagos/email/deploy que compila a la primera) sobre el cual iterar.
Self-hosters / privacy-first	Generación 100% local sobre Ollama: cero datos saliendo de tu máquina, cero costo por token.
Investigadores de sistemas multi-agente	Un banco de pruebas open-source de evolución genética, torneos adversariales y aprendizaje colectivo.

Diferenciadores

1. **Determinístico donde importa, generativo donde aporta.** El spine se *copia y configura*, nunca se alucina ([README.md](#) , [AGENTS.md](#) §2).

2. **Genético, no monolítico.** Tres equipos con filosofías opuestas divergen y compiten en un torneo con jurado multi-modelo ([web/lib/tournament.ts](#)).
3. **Evoluciona.** Diario de agentes con embeddings, XP/niveles, y skills entrenables que sólo se aceptan si baten un held-out set ([web/lib/progression.ts](#) , [web/lib/skill-evolution.ts](#)).
4. **Self-healing real.** Un *runtime gate* que bootea la app y la smoke-testea — el que en Stayforge expuso **7 bugs reales invisibles a un compile verde** ([web/scripts/build-local.mjs](#)).
5. **Local-first y BYO-credentials.** Sin lock-in de proveedor; un único chokepoint `call()` con dos protocolos ([web/lib/openai.ts](#)).

Mission

Misión

Colapsar la distancia entre una idea de SaaS y un producto desplegable y monetizable, sin sacrificar la confiabilidad de ingeniería que separa un demo de un negocio. Puglit existe para que la parte que jamás debe romperse (webhooks de Stripe, auth, migraciones de DB, deploy) se ensamble mecánicamente y a la primera, liberando a la IA — y al humano — para concentrarse en lo único que es de verdad único: la lógica de negocio.

Visión a 10 años

Que generar software de aplicación deje de ser un acto de *escritura* y pase a ser un acto de *evolución y selección*. En la visión a una década, el "Module Genome" (el catálogo viviente de [web/lib/module-registry.ts](#)) y el "brain" colectivo ([web/lib/brain-sync.ts](#) , tablas append-only tipo CRDT) acumulan suficiente conocimiento verificado como para que la mayoría de los productos converjan a algo correcto sin intervención humana — y para que cada build que corre en cualquier máquina del mundo deje a la siguiente generación de agentes un poco más capaz que la anterior.

North Star Metric

% de builds que pasan el runtime gate ([build-local.mjs](#)) en el primer intento, sin reparación humana — la "first-pass yield" del factory.

Es la métrica correcta porque captura simultáneamente generación, validación, gobernanza y conocimiento (las cuatro preocupaciones que [AGENTS.md](#) §1 separa). Un sistema que genera mucho código pero no converge "vale cero" ([AGENTS.md](#) §3.3); el North Star castiga precisamente eso. La litmus test de **Stayforge** — 80 archivos, 8 tablas, anti-double-booking con `EXCLUDE USING gist` , pricing determinístico en centavos enteros — es el tipo de build contra el que esta métrica se mide.

Core Principles

Estos siete principios son **innegociables**. Cuando un nuevo feature los contradice, el feature pierde.

Agent First

Puglit no es una librería con un agente atornillado encima; es un **sistema de agentes** desde la raíz. El roster son **25 roles × 3 equipos = 75 agentes** (`web/lib/roster.ts`), cada uno una identidad persistente con stats RPG `{creativity, rigor, security, speed, depth}` que **manejan los parámetros de Ollama** — `tempFromStats()` traduce stats a temperatura (roles creativos corren calientes, roles rigurosos corren fríos). La cognición está distribuida entre arquitectos (master-spec, contracts, domain), ingenieros (backend, frontend), QA/confiabilidad, seguridad, completeness-critic, researcher, reference-studier y la queen-bee. El humano describe; los agentes deciden, divergen, critican y convergen.

Open Source

Core MIT (`LICENSE`, `README.md`). No hay "edición enterprise" oculta detrás del moat: el moat es el conocimiento acumulado y el motor de evolución, no el código fuente retenido. Los módulos creados por el swarm se reflejan a `modules/<name>/` para *git review* (`web/lib/module-registry.ts`), de modo que el crecimiento del sistema es auditable en abierto.

Self Hosted

Todo corre en tu infraestructura. La referencia es una caja **RunPod A40 48GB** con **Postgres 14 + pgvector** y **Ollama (~77GB de modelos)** (`_SHARED_FACTS.md`, sección Infrastructure). Los sidecars (p. ej. ScrapeGraphAI, `infra/scrapegraph-sidecar.py`) también son self-hosted contra el Ollama local. No hay un servicio central de Puglit del que dependas para generar.

BYO Credentials

Un único chokepoint, `call()` en `web/lib/openai.ts`, con **dos protocolos**: `openai` (OpenAI / Gemini / Ollama / custom) y `anthropic`. Los tiers de modelo (`premium` para arquitectura/judging/review/SkillOpt, `balanced`, `cheap` para extracción, `code` para routes/pages/repair) se mapean por env `PUGLIT_MODEL_*`, así que **traés tus propias claves y mezclás proveedores por tier**. El default es 100% local sobre Ollama: sin clave, sin costo.

Deterministic over Magic

El principio rector, citado verbatim de `AGENTS.md` §2: "*Deterministic spine + LLM swarm on the bespoke 20%*." La parte que debe ser correcta y aburrida se ensambla mecánicamente; la superficie de rediseño está **acotada por contratos** ("Contracts are the single source of truth"). Los ~85 módulos se inyectan vía `deterministicX(config, bp) → {files, extraSql}` cuando matchean keywords (`web/lib/module-registry.ts`), nunca se improvisan. La parametrización al nombre real de la tabla detectada (p. ej. `rentals-module.ts`, *zero hardcoding*) es la materialización de este principio: lógica probada, cero alucinación.

Local First

El camino feliz no requiere internet ni una tarjeta de crédito. `README.md`: "*Run it locally (100% local, no paid keys)*". El wiring se verifica con `GET /api/doctor?ping=1`. La persistencia del brain prioriza el Postgres local/cloud como autoritativo, con snapshots a git/bucket (`infra/brain-snapshot.sh` / `brain-restore.sh`). Lo local no es un modo degradado: es el modo por defecto.

Evolutionary Software

El software no se escribe una vez: **evoluciona**. Tres mecanismos lo sostienen (`web/lib/progression.ts` , `web/lib/skill-evolution.ts`):

- **Diary.** Lecciones, wins y críticas con quality 0–10 y embeddings `nomic-embed` en `puglit_agent_diary` . `relevantLessons` aplica piso de relevancia 0.35 + decay por recencia + $quality \geq 45 + outcome \neq failure$ (**anti-poisoning**: el sistema no aprende de sus propios fracasos).
- **XP.** $XP = area \times victory \times participation \rightarrow niveles$.
- **SkillOpt** (microsoft/SkillOpt): el skill DOC de cada rol es **estado entrenable**; un optimizer propone ediciones acotadas (add/delete/replace) aceptadas **sólo si baten al actual en un validation set congelado** (5 tareas → blueprint rollout → `objectiveScore`), con buffer de ediciones rechazadas.

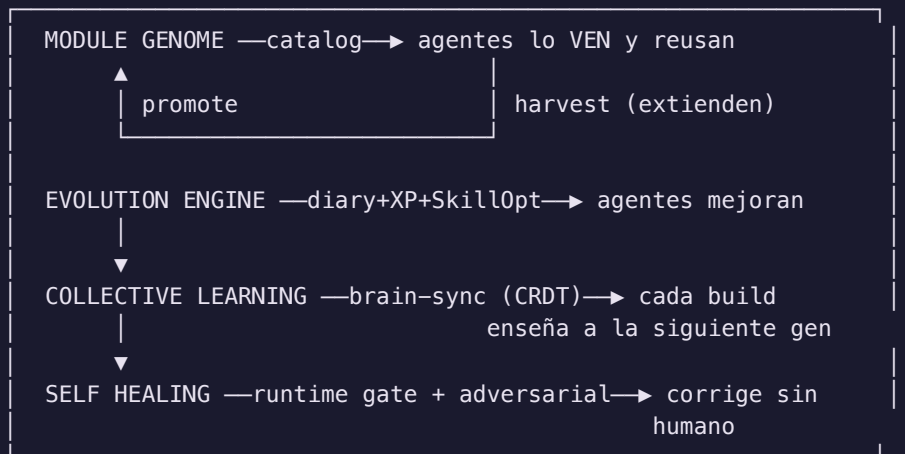
What Puglit Is Not

Definir el moat exige definir las fronteras. Puglit **no** es ninguna de estas cosas:

No es...	Por qué no	Evidencia en el código
Un CRUD builder	No mapea tablas a formularios. El blueprint agent <i>infiere la forma funcional real</i> del producto (operaciones, engine, invariantes), no un scaffolding genérico. La litmus test Stayforge incluye anti-double-booking con <code>EXCLUDE USING gist</code> y reviews double-blind — eso no es CRUD.	<code>web/lib/app-builder.ts</code> (header) · <code>_SHARED_FACTS.md</code> (Proof points)
No-code tradicional	La salida es un repo Next.js + Postgres real , código que se lee, se versiona en git y se despliega — no un runtime propietario detrás de un editor visual.	<code>README.md</code> (stack fijo) · <code>spine/</code>
Un wrapper de ChatGPT	El LLM toca sólo el 20% bespoke; el 80% es determinístico. Y hay tres equipos compitiendo con jurado multi-modelo, evolución genética, runtime gate y revisión adversarial — no un único prompt a una única API.	<code>AGENTS.md</code> §2 · <code>web/lib/tournament.ts</code> · <code>web/lib/adversarial-review.ts</code>
Un IDE	No es una herramienta donde un humano escribe código asistido. Es un <i>factory</i> : describís una idea y los agentes producen el sistema completo. El humano no está en el loop de tecleo.	<code>README.md</code> ("Describe your idea → ... → complete SaaS")

Strategic Moats

El moat de Puglit no es el código de generación (eso es, en palabras de `AGENTS.md` §2, "*a commodity*"). El moat son cuatro activos compuestos que se profundizan con cada build.



Module Genome

`web/lib/module-registry.ts` es un **directorío viviente** de ~85 módulos con un ciclo de vida explícito:

- **Catalog** — `moduleCatalog()` se inyecta en los prompts: los agentes *ven* lo que existe y lo reusan.
- **Harvest** — `harvestModules()` registra connectors/integrations reutilizables que un build produjo: el directorío *crece*.
- **Promote** — `experimental → candidate → stable → core`: las mejoras/fixes fluyen de vuelta.

Hay un **grafo de dependencias duras** (`MODULE_REQUIRES`, p. ej. `billing → crypto`, `rag → llm`, `agent → llm, crypto`) con cierre transitivo (`dependencyClosure`), de modo que un keyword que dispara un módulo dependiente nunca despacha un build roto sin su base. Este genoma es acumulativo: cuanto más se usa Puglit, más rico es el catálogo del que parte el siguiente build.

Evolution Engine

Detallado en *Core Principles → Evolutionary Software*: el diario con embeddings anti-poisoning, XP/niveles, y SkillOpt con validación held-out. Es un moat porque el conocimiento es **verificado** (sólo entra lo que bate a la baseline en tareas congeladas) y **persistente** (sobrevive entre proyectos en `puglit_agents` / `puglit_skills`, ver `web/sql/genetic.sql`).

Collective Learning

`web/lib/brain-sync.ts` hace del aprendizaje un bien colectivo: tablas append-only se unen como **grow-set CRDT** (nunca se pierde un exemplar), y los skills mutables se arbitran por `val_score`, con el Postgres cloud como autoritativo más snapshots a git/bucket. El resultado: cada máquina que corre Puglit puede contribuir y consumir del mismo brain, y el conjunto mejora más rápido que cualquier instancia aislada.

Self Healing

Tres capas que corrigen sin intervención humana:

- **Static + repair** — `web/lib/swarm-checks.ts` (escaneo de secrets/SQLi/phantom-table) y `web/lib/swarm-repair.ts` (repara phantom-tables infiriendo el schema de la intención, con escalada a frontier).

- **Adversarial review** — `web/lib/adversarial-review.ts` : tres lentes ortogonales (Auditor=lógica, Adversary=abuso/seguridad, Pragmatist=diseño) → cross-examination determinística (≥ 2 lentes = confirmado) → veredicto **SHIP / SHIP-WITH-CAVEATS / BLOCK** + reparación acotada.
- **Runtime gate** — `web/scripts/build-local.mjs` : **bootea la app de verdad**, smoke-testea las páginas (0 errores 5xx), corre vitest + coverage, y somete la evidencia a una *Queen review*. En Stayforge expuso **7 bugs reales invisibles a un compile verde**. Es el moat más difícil de copiar: probar que el código *corre*, no sólo que *compila*.

Long-Term Roadmap

El estado actual (`README.md`) es: **Phase 1 — extracción del Spine** en curso, con CLI (`npx create-puglit`) y Web (puglit.com) como fases siguientes. El roadmap de visión:

v1 — The Factory (presente / cercano)

- Spine determinístico completo: auth, rate-limit, analytics, i18n, mailer, db pool (`spine/`).
- Swarm genético operativo: 75 agentes, torneo de 3 equipos, ~85 módulos, runtime gate.
- Local-first sobre Ollama + BYO providers.
- **Hito de prueba en curso:** el *100-build brain-training batch* (`infra/train-brain.sh` + `brain-training-ideas.tsv`) — apps diversas (juegos retro, calculadoras, trackers, verticales de negocio, money, catálogo, quiz) para pre-cargar el brain (diary/exemplars/metrics/skills) y snapshotarlo a git.

v2 — Distributed Evolution

- Roster que escala más allá de 1 GPU a **workers paralelos + graph DB** (el límite de hoy es de hardware, no de diseño — `web/lib/roster.ts` lo dice explícitamente: "75 definiciones reales, no 75 procesos simultáneos").
- Brain colectivo federado entre instancias self-hosted vía `brain-sync` (CRDT), con genoma de módulos compartido.
- SkillOpt continuo sobre validation sets más grandes y diversos.

v3 — Self-Designing System

- El propio sistema propone y promueve nuevos *roles* y *módulos* (no sólo nuevos skills dentro de roles existentes), cerrando el loop catalog→harvest→promote a nivel del roster mismo.
- Convergencia mayoritaria sin intervención humana, medida por la North Star Metric (first-pass yield del runtime gate).

Moonshots

- **Verticals que se auto-descubren:** que el sistema, viendo el brain colectivo, proponga verticales de mercado enteros (al estilo `rentals-module.ts`) antes de que nadie los pida.
- **Generación de evidencia de negocio, no sólo de código:** que cada app salga no sólo desplegable sino con un caso de monetización instrumentado de fábrica.
- **Un brain que enseña a humanos:** que el diario de lecciones verificadas se vuelva un corpus consultable de cómo construir SaaS correctamente — el sistema documentando su propia

disciplina.

Síntesis filosófica. Todo lo anterior se reduce a una sola apuesta: *el software de aplicación debe ensamblarse donde es mecánico y evolucionar donde es único*. Los siete principios protegen esa apuesta; los cuatro moats la componen en el tiempo; la North Star la mide. El resto de este whitepaper muestra cómo el código la cumple.